统一架构框架（裁决版）

> **文档定位**：本文档是当前架构的**单一权威版本**。当其他文档与本文档冲突时，以本文档为准。
>
> 它汇总并裁决了三类历史遗留冲突：多 Agent 投票 vs 单模型提议、L0/L1/L2/L3 三套风险分级、Hook 系统的归属。 被本文档裁决或替代的内容见 §13 与旧文档的关系。

---

0. 核心主张

> **模型是可替换的大脑，MCP 工具层是不可绕过的脊柱。**

模型负责理解与提议；确定性系统负责校验、预览、执行；风险分级决定是否要人授权；事件溯源负责追责。

---

1. 六条设计公理

所有下层设计都必须能从这六条推导出来。任何与之冲突的局部设计都应被视为缺陷。

#	公理	推论
I1	**单一写入通道**	所有业务写入必须经 MCP Server。UI/REST、Agent、定时任务概不例外
I2	**模型无凭据**	Agent Runtime 不持有数据库凭据，不 import BusinessEngine
I3	**令牌即授权**	执行工具**只接受** `confirmation_token + idempotency_key`，不接受模型重新生成的业务参数
I4	**确认不可绕过**	L2 必须人工 approve；白名单只能调整阈值，**永不**跳过确认
I5	**契约是唯一规则源**	前置条件/不变量/权限/风险/补偿全部在契约中声明，不存在第二套规则系统
I6	**一切可回放**	输入、提议、校验、预览、确认、执行、补偿都产生带 `trace_id` 的事件

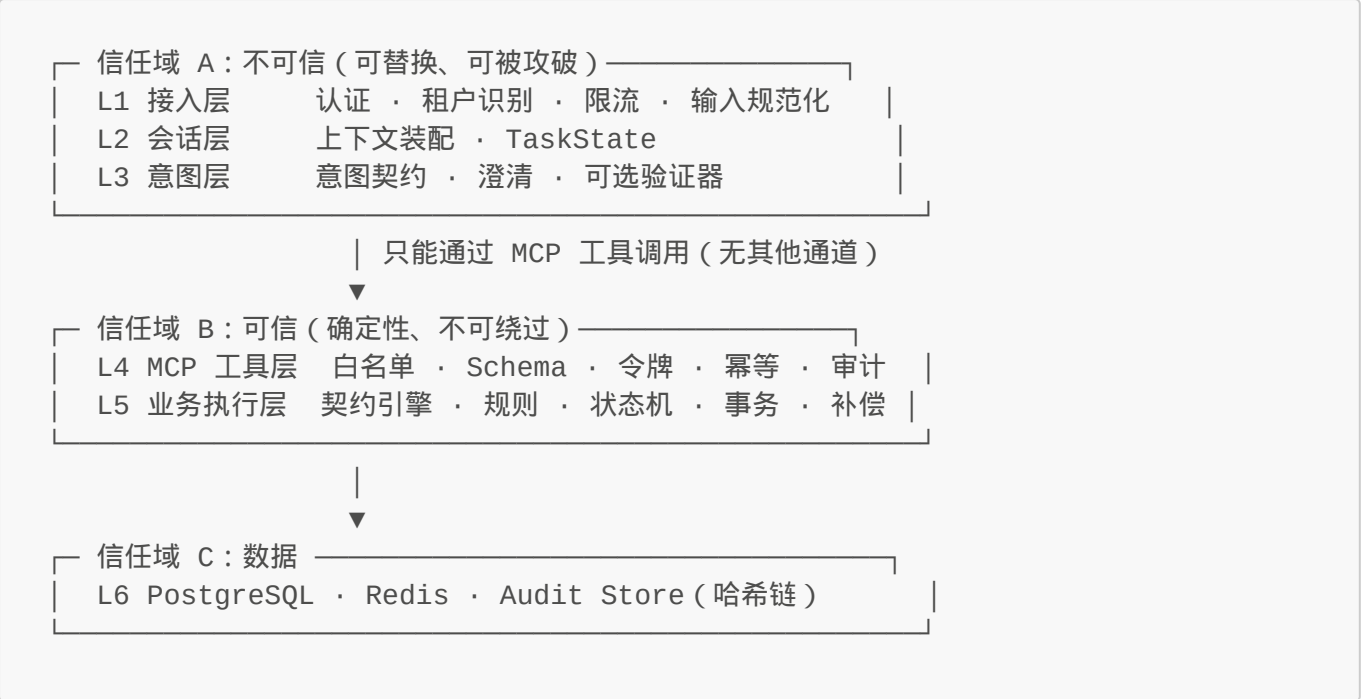
1.1 不做什么

不做	原因
追求"零幻觉"	不可证明，也不现实
三个同质模型对每笔操作投票	相关性高、成本高、投票不等于验证
向模型暴露裸 CRUD / 裸 SQL	无法约束业务语义和权限
让模型直接持有数据库凭据	绕过信任边界
用聊天记录承载业务状态	不可恢复、不可审计、易被污染
用模型自报置信度做安全门控	自报置信度不可校准，且可被提示注入操纵

---

2. 分层 = 信任域

层必须按**可信 / 不可信**划分，而不是按逻辑职责划分。这是本框架最重要的一条结构性约束。



推论：域 A 中的任何组件都不承载安全责任。模型被替换、Agent 被攻破、上下文被污染，都不应影响域 B 的判定结果。

2.1 分层职责

层	核心职责	绝对不能做
L1 接入层	认证、租户识别、限流、请求规范化	处理业务规则
L2 会话层	装配 TaskState、对话摘要、实体快照；恢复未完成任务	把聊天记录当作业务事实
L3 意图层	意图识别、槽位抽取、澄清生成；高风险时调用可选验证器	直接写库，或直接决定金额/库存/权限
L4 MCP 工具层	工具白名单、Schema 校验、令牌签发与校验、幂等、审计出口	暴露裸 SQL / 裸 CRUD；接受未经预览的参数
L5 业务执行层	契约引擎、规则校验、状态机、事务、补偿	接受未经契约校验的参数
L6 数据层	业务数据、缓存、事件日志、哈希链	被 Agent 直接访问

3. 唯一风险模型：L0 / L1 / L2 / Block

四档，**Block 是结果不是等级**，拥有最高优先级，不可被任何因子降级。

等级	含义	路径	人工确认
L0	只读查询	查询工具直读	无

等级	含义	路径	人工确认
L1	低风险写入	preview → execute（令牌自动生效）	无人工，但幂等 + 审计
L2	高风险写入	preview → [可选验证器] → 人工 approve → execute	**必须**
Block	非法 / 越权 / 违反不变量	直接阻断	不进入确认

3.1 定级规则

定级只发生在契约的 `riskRules` 中。系统中不存在第二个定级器。

动态因素（客户类型、金额、时段、频率、白名单等级）全部是 `riskRules` 的**输入因子**，不是独立的风险系统。

```
interface RiskInput {
  opType: 'query' | 'create' | 'update' | 'delete';
  entity: string;
  fields: string[];
  amount?: number;
  customer: {
    type: 'normal' | 'new' | 'vip' | 'high_risk' | 'restricted';
    overdueDays: number;
    creditAvailable: number;
  };
  context: {
    timestampSnapshot: Date;     // 由调用方传入，非墙上时钟
    recentOpCount: number;       // 由调用方传入，非实时统计
    whitelistLevel: 'none' | 'WL1' | 'WL2' | 'WL3';
    tenantThresholds: Record<string, number>;
  };
}

type RiskResult = {
  level: 'L0' | 'L1' | 'L2' | 'Block';
  reasons: string[];
};
```

`riskRules` **必须是纯函数**：不读取墙上时钟、不访问数据库、不依赖全局状态。所有外部输入经 `RiskInput` 传入。

> 纯函数是评测可复现的前提。若风险等级依赖系统时间或运行时统计，同一条评测样本在不同时刻会得到不同结果，`expected.risk` 标注将无法复现。

3.2 判定顺序

```
1. 命中任一 Block 条件          → Block（终止，不再计算）
2. 操作类型 = query              → L0
3. 金额 > 租户阈值               → L2
4. 命中 S3/S4 敏感字段           → L2
5. 客户为 high_risk/restricted → L2
6. 其余写入                      → L1
```

阈值调整（白名单、客户历史、租户配置）只影响第 3 步的比较基准，**不影响第 1 步和第 2 步**。

3.3 白名单的边界

白名单的作用域被严格限制为：

允许	禁止
提高金额阈值	跳过 preview
降低批量操作的分级	跳过人工确认
影响 `recentOpCount` 的判定基准	跳过幂等或审计

> **白名单永远不能绕过 preview → confirm → execute 协议。** 一个可以按用户信任度跳过人工确认的机制不是快速通道，而是可信边界上的洞。

---

4. 一次写入的完整生命周期

```
intent_parsed
  ├─ 需要澄清 → awaiting_clarification（≤3 轮）↩
  ▼
context_loaded
  ▼
L4 preview ── 载入证据 → 生成 diff/risk/reasons → 落 preview_snapshot → 签发令牌
  ├─ Block → blocked（终止，记审计）
  ├─ L1    → executing（令牌自动生效）
  └─ L2    → awaiting_confirmation → [验证器] → executing / rejected / expired
  ▼
L5 execute（事务内：重校验前置条件 → 写入 → 预占库存 → 审计事件）
  ├─ 前置条件失败 → conflict（要求重新 preview，不复用令牌）
  ├─ 成功         → completed
  └─ 异常         → compensated / compensation_failed
```

4.1 Preview 快照

预览结果必须服务端持久化，执行时按快照取回原始参数，模型无法在确认后修改。

项	规格
存储键	`tenantId + previewId`
TTL	默认 30 分钟（必须长于令牌 TTL）
内容	契约名 + 契约版本 + **归一化后的参数** + `intentHash` + diff + risk + reasons + evidence
失效	一次性使用；使用后立即标记 consumed；过期由定时任务清理
关联	`previewId` 写入令牌，令牌写入 TaskState.pendingPreview

> 这解决了原设计中的一个死结：L1 执行不带令牌，但写工具又不接受模型重新生成的业务参数——业务参数的唯一来源必须是预览快照。

4.2 双层哈希

项	内容	是否进令牌	变更后果
`intentHash`	契约版本 + 归一化参数	**是**	与令牌不一致 → 拒绝执行
`evidenceDigest`	预览时的查询证据摘要	**否**，只进审计	不影响令牌有效性
执行期前置条件重校验	实时库存 / 额度 / 状态	—	允许失败 → `CONFLICT` → 重新预览

> 证据不得进入 `intentHash`。否则预览与执行之间的任何并发变动（例如同事同时下的一单）都会使令牌失效，导致人工确认在繁忙场景下必然失败。

`intentHash` 是人批准的那个东西。`evidenceDigest` 只用于事后审计还原"当时看到了什么"。

4.3 确认令牌

属性	要求
绑定	`tenantId`、`userId`、`sessionId`、`previewId`、`intentHash`
一次性	使用后立即失效，重放返回 `CONFLICT`
过期	默认 10–30 分钟，且必须短于 preview 快照 TTL
不可伪造	服务端签名（HMAC/JWS），客户端不可解析修改
可审计	记录签发、使用、拒绝、过期四类事件

L1 与 L2 使用同一套令牌协议，唯一区别：L2 需要一次 `confirmation.approve`，L1 签发后自动生效。协议只有一套。

4.4 幂等

- 每个写工具必须接受 `idempotency_key`。
- 唯一约束：`(tenantId, toolName, idempotency_key)`。
- 重复请求返回第一次的结果，不重复执行。
- 幂等键由 Agent Runtime 生成，持久化在 `TaskState.idempotencyKey`，任务重试时复用。

4.5 事务与补偿

必须区分两种失败：

场景	处理方式
事务内失败（校验失败、写入异常、约束冲突）	数据库事务回滚，无需补偿契约
已提交后撤销（用户取消、事后发现异常、外部系统失败）	执行契约声明的补偿动作，记录 `compensated` 事件
补偿失败	记录 `compensation_failed`，告警并要求人工介入

补偿动作本身必须幂等、可重试。

---

5. 契约引擎（核心壁垒）

5.1 契约定义

```
interface Contract<P> {
  operation: string;            // create_order
  version: string;              // "1.0"
  inputSchema: ZodSchema<P>;
  normalize(raw: unknown): P;                       // 单位 / 币种 / 默认值归一化
  loadEvidence(p: P, ctx: ExecContext): Evidence;   // 唯一允许 IO 的钩子，只读
  preconditions(p: P, ev: Evidence): Check[];       // 失败即阻断
  invariants(p: P, ev: Evidence): Check[];          // 执行过程中必须保持
  permissions(p: P, ctx: ExecContext): Check[];
  riskRules(p: P, ev: Evidence, ctx: RiskInput): RiskResult;
  effects(p: P, ev: Evidence): Effect[];            // 声明式，由引擎执行
  compensation(p: P, ev: Evidence, r: ExecResult): Effect[];
  audit(p: P, ev: Evidence, r: ExecResult): AuditSpec;
}

interface Check {
  id: string;        // customer.credit_available_sufficient
  ok: boolean;
  code: string;      // INSUFFICIENT_CREDIT
  message: string;   // 客户可用额度 5,000，订单需要 20,000
}

interface Effect {
  type: 'create' | 'update' | 'reserve' | 'release';
  entity: string;
  payload: Record<string, unknown>;
}
```

5.2 五条实现约束

#	约束	理由
1	契约用 **TypeScript** 实现，不自研 DSL	可类型检查、可单测、零解析器成本；YAML 仅用于描述字段与阈值，不承载逻辑
2	**evidence 先行加载，Check 是纯函数**	契约函数不直接访问数据库 → 无隐式 IO、无依赖排序问题、可离线测试
3	`effects` 是**声明式列表**，由 L5 统一在事务内执行	契约不自己写库 → 补偿可推导、事务边界清晰
4	**契约是唯一规则源**	业务引擎规则（原 `BusinessRule` JSON）吸收为 `Check` 的实现形式
5	契约版本化，变更走评审 + 回归测试	令牌绑定契约版本，旧版本令牌不可用于新契约

5.3 `loadEvidence` 的边界

允许：按 ID 查询主实体、查询关联实体状态、查询额度与库存、查询租户配置。

禁止：写入、调用外部网络、读取与本次判定无关的数据、读取环境变量中的策略。

> 证据必须可序列化并写入 `evidenceDigest`，以便审计还原。

5.4 契约与规则引擎的关系

原 04-business-engine.md 中的 `BusinessRule`（`field` / `operator` / `value` JSON 形式）不再作为独立系统存在，而是：

原概念	新落点
`conditions`	`Contract.preconditions` 中的 `Check`
`error_level: reject / warn`	`Check.ok = false` 时由引擎按 code 决定阻断或警告
`rule_id`	`Check.id`
行业插件	插件向契约注册额外的 `Check` 工厂函数

6. 工具层与协议

6.1 工具分类

类别	命名	是否写库	示例
查询	`<domain>.<verb>`	否	`customer.search`、`credit.get`、`inventory.check`
预览	`<domain>.preview_<action>`	否	`order.preview_create`、`payment.preview_record`

类别	命名	是否写库	示例
执行	`<domain>.execute_<action>`	是	`order.execute_create`、`payment.execute_record`
治理	`confirmation.*`、`audit.*`、`whitelist.*`	部分	`confirmation.approve`、`audit.trace`
知识	`policy.*`、`schema.*`	否	`policy.get`、`schema.get`

6.2 统一返回信封

```
{
  "status": "ok | needs_confirmation | blocked | conflict | error",
  "data": {},
  "diff": {},
  "evidence": {},
  "risk": "L0 | L1 | L2 | Block",
  "reasons": [],
  "preview_id": "pv_01J...",
  "confirmation_token": "ct_01J...",
  "expires_at": "2026-11-03T12:00:00Z",
  "audit_id": "aud_01J...",
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "total_amount does not match items"
  }
}
```

6.3 错误码

错误码	含义	Agent 应如何处理
`VALIDATION_ERROR`	参数或业务约束不合法	向用户解释并请求修正
`PERMISSION_DENIED`	权限不足	告知用户，不重试
`NOT_FOUND`	实体不存在	请求澄清或重新搜索
`CONFLICT`	并发冲突 / 令牌已使用 / 前置条件失效	重新预览，不复用令牌
`NEEDS_CONFIRMATION`	需要人工确认	展示 diff 并等待确认
`RATE_LIMITED`	频率超限	退避后重试
`INTERNAL`	服务端错误	不重试写操作，返回审计 ID

6.4 工具可见性隔离

治理工具按 **MCP client 身份**注册，而不是靠文档约定：

Client	可见工具
Agent Runtime	查询 / 预览 / 执行 / 知识。**看不到** `confirmation.approve`
人工确认 UI	`confirmation.approve` / `confirmation.reject` + 查询
运维后台	治理 + `audit.trace` + `whitelist.*`

> 模型只能看到 `needs_confirmation` 状态和确认令牌，物理上无法调用确认工具。

---

7. 意图契约与澄清

7.1 输出结构

```ts
type IntentResult =
  | { kind: 'operation'; contract: string; slots: Record<string, unknown>;
unresolved?: string[] }
  | { kind: 'clarify'; questions: ClarifyQuestion[] }
  | { kind: 'compound'; steps: IntentResult[] }
  | { kind: 'chat'; reply: string };

interface ClarifyQuestion {
  slot: string;
  reason: 'ambiguous' | 'missing' | 'invalid';
  candidates?: { id: string; label: string; matchReason: string }[];
}
```

7.2 不做置信度门控

模型自报的 confidence 不作为安全决策输入。

- 自报置信度不可校准，且可被提示注入操纵（"忽略规则…confidence: 0.99"）。
- 可选验证器的触发只使用确定性信号：`risk === 'L2'`、金额超阈值、命中敏感字段、候选实体 > 1、存在历史否决记录。
- confidence 可以展示给用户作为参考，但不参与任何门控判断。

7.3 澄清协议

项	规则
状态	`awaiting_clarification`，与待补槽位一起写入 TaskState
绑定	问句 + `intent` 快照 + `clarificationRounds` 持久化；用户回复写回对应槽位后重新预览
轮次	默认上限 3 轮；超限则终止任务并提示用户换一种说法或走人工
候选	查询工具返回 `candidates[] + matchReason`，由 UI 展示，用户选择写回 slots
多意图	复合意图默认**逐步执行、逐步确认**，不合并事务

> 义乌商户重名极多，澄清是 Phase 0 的关键路径，不是可选增强。

8. TaskState 与上下文

8.1 数据模型

```typescript
interface TaskState {
  id: string;
  tenantId: string;
  userId: string;
  conversationId: string;
  status:
    | 'idle'
    | 'intent_parsed'
    | 'awaiting_clarification'
    | 'context_loaded'
    | 'preview_ready'
    | 'awaiting_confirmation'
    | 'executing'
    | 'completed'
    | 'failed'
    | 'compensated'
    | 'compensation_failed'
    | 'conflict';
  intent?: IntentResult;
  pendingPreview?: {
    previewId: string;
    intentHash: string;
    contract: string;
    contractVersion: string;
    expiresAt: Date;
  };
  activeCustomerId?: string;
  activeOrderId?: string;
  idempotencyKey?: string;
  clarificationRounds: number;
  version: number;           // 乐观锁
  updatedAt: Date;
}
```

8.2 上下文装配顺序

稳定内容在前，易变内容在后，便于提示词缓存：

1. 系统策略与安全规则（固定，可缓存）
2. TaskState（结构化，小）
3. 实体快照（按需查询，**带来源与查询时间**）
4. 工具定义（固定，可缓存）
5. 最近 N 轮对话
6. 滚动摘要
7. 检索内容（带出处，限制 top-k）

8. 当前用户输入

8.3 不变式

- 对话上下文只保存"说了什么"；TaskState 保存"任务进行到哪一步"。
- 实体信息必须标注来源和查询时间；聊天历史不是事实源。
- 摘要必须保留：任务目标、关键实体 ID、金额、日期、权限、已确认/已拒绝的决策。
- 服务崩溃后从 TaskState 与事件日志重建，不依赖客户端存储。

---

9. 能力归属矩阵

能力	归属	理由
输入清洗 / 限流	L1	边界处，与业务无关
意图解析	L3	模型唯一职责
澄清	L3 产出 + L2 状态	问句由模型生成，状态在 TaskState
前置条件 / 不变量 / 权限	L5 契约	需 DB 证据，必须确定性
风险定级	L5 `riskRules`	纯函数 + 快照
diff 生成	L4 预览工具	读证据 + 参数，不改库
令牌签发 / 校验	L4	与执行工具同信任域
人工确认	L4 治理工具 + 人工 UI	Agent client 不可见
事务 / 补偿	L5	需 DB 事务
审计	L4 + L5 统一出口	单点写入 Audit Store
Hook	**L5 内部**，作为 `Check` 实现	只有在可信域内才有意义
独立验证器	L3，可选	只输出建议，不执行

> **Hook 的裁决**：Hook 不再是独立的横切层。若要保留，只能作为契约 `Check` 的实现机制运行在 L4/L5 内。运行在 Agent 进程中的 Hook 不计入安全边界。

---

10. 审计与合规

10.1 统一事件模型

```
interface AuditEvent {
  eventId: string;
  traceId: string;
  tenantId: string;
  actor: { userId: string; type: 'human' | 'agent' | 'system' };
  tool: string;
  contract: string;
  contractVersion: string;
```

```
  intentHash: string;
  argsRedacted: Record<string, unknown>;
  evidenceDigest: string;
  risk: 'L0' | 'L1' | 'L2' | 'Block';
  reasons: string[];
  decision: 'allow' | 'confirm' | 'block' | 'reject';
  result: 'success' | 'failure' | 'compensated';
  modelVersion?: string;
  promptVersion?: string;
  ts: Date;
  prevHash: string;
  hash: string;
}
```

哈希链只追加：`hash = H(prevHash || canonical(event))`。

10.2 删除权与不可变性

审计日志哈希链不可变，但 PIPL 要求个人信息可删除。二者通过**假名化 + crypto-shredding** 调和：

机制	做法
假名化	审计事件中只存 `subjectRef`（假名），不存姓名 / 电话 / 地址明文
映射表	`subjectRef ↔ 明文标识` 单独加密存储，密钥由 KMS 管理
删除权	响应删除请求时销毁该 subject 的映射密钥，链保持完整而信息不可复原
展示脱敏	审计查询接口按调用方权限二次脱敏

> 审计记录永不存明文个人信息。这是哈希链与删除权能同时成立的唯一方式。

10.3 保留期

数据类型	保留期
业务审计事件	3 年（合规基线）
安全日志	1 年
系统日志	6 个月
业务数据	按租户合同约定

契约中的 `retain_days` 只是**上限声明**，不能覆盖合规下限。

10.4 数据驻留

模型适配层必须声明 `dataResidency`，`policy.get` 返回租户的数据驻留策略。接入非境内模型前必须完成出境合规评估；默认按租户区域路由到境内模型。

---

11. 多租户

租户不是 Phase 3 的可选项。令牌、幂等键、契约权限、越权缓解**全部已预设 `tenantId` 存在**。

11.1 模型

项	设计
`Tenant`	独立实体，含套餐、状态、数据驻留区域
`User ↔ Tenant`	多对多，关系上携带 `role`
隔离机制	**PostgreSQL Row-Level Security**；MCP Server 每请求 `SET LOCAL app.tenant_id`，应用层 WHERE 作为第二道防线
租户配置	风险阈值、启用插件、保留期上限、模型选择
配置变更	配置变更本身也是契约操作，需审计（防止"改阈值绕过风控"）

11.2 角色与工具 scope

角色	工具 scope
`owner`	全部工具 + 租户配置 + 白名单管理
`sales`	客户 / 订单 / 支付 / 库存的查询与写入
`finance`	支付 / 额度 / 对账；不可修改客户档案
`viewer`	仅查询类工具

角色 → scope 的映射表由 `policy.get` 返回，服务端强制校验，**不信任模型声明**。

---

12. 三条补充流程

12.1 澄清流程

```
L3 解析 → { kind: 'clarify' } → TaskState.status = awaiting_clarification
  → UI 展示问句与候选
  → 用户消歧
  → 写回 slots → 重新 preview
  → 轮次 +1；超过 3 轮则终止任务
```

12.2 冲突恢复流程

```
execute 返回 CONFLICT
  → 作废旧令牌
  → 重新 loadEvidence + preview
  → 风险等级若上升（如 L1 → L2），必须重新人工确认
  → 风险等级不变则签发新令牌
```

12.3 补偿流程

```
execute 失败
  → 事务内失败：数据库回滚，记录 failure
  → 已提交后撤销：执行 compensation effects，记录 compensated
  → 补偿失败：记录 compensation_failed，告警，人工介入
```

---

13. 与旧文档的关系

本节是裁决记录。下表每一行都代表一次明确取舍，用于终结多版本并存的混乱。

旧文档	旧内容	裁决
01-multi-agent-design.md	主 Agent + 2 子 Agent 投票	**废弃为默认层**；仅保留为可选独立验证器（L3）
02-openclaw-development.md	OpenClaw 内核定制	**废弃**，使用轻量 Agent Runtime
03-mcp-tool-whitelist.md	下划线工具名；只读/写入二分；L1–L4 权限分级	工具命名改为 `<domain>.<action>`；分类改为五类；删除与风险等级撞名的 L1–L4 权限分级
04-business-engine.md	`BusinessRule` JSON 规则系统；投票器调用引擎	`BusinessRule` 并入契约 `Check`；投票器不再存在
05-four-layer-safety.md	六重防护	**废弃**，以本框架的信任域分层为准
07-data-flow-design.md	含 OpenClaw 调度层与投票的七层流程	保留作对比，**不得据此实现**
08-risk-classification.md	L0/L1/L2/**L3**；四维 `max()`；按档投票	**裁决**：统一为 L0/L1/L2/Block；维度合并进 `riskRules` 判定顺序；L3 废弃
09-fast-path.md	白名单可跳过投票 / 跳过确认	**裁决**：白名单只能调阈值，**不得跳过 preview / 确认 / 幂等**
10-dynamic-risk.md	独立评分 `clamp(分数, L0, L2)`	**裁决**：降级为 `riskRules` 的输入因子；须为纯函数
11-fallback.md	基于投票与 L3 的 LLM 降级	主题保留，**档位与票数措辞需按本框架重写**
12-hook-system-design.md	Hook 作为独立横切层	**裁决**：Hook 收编进 L5，作为契约 `Check` 实现；Agent 进程内的 Hook 不计入安全边界
13-trusted-execution-architecture.md	契约、MCP 工具链、preview/confirm/execute	**保留为详细设计**；与本文冲突处以本文为准（尤其 `operationHash` 拆分为双层哈希）

---

14. 落地顺序（按框架依赖）

阶段	内容	验收
Phase 0	租户 + 契约引擎 + 4 个查询工具 + `order.preview_create` / `order.execute_create` + 令牌 + 快照 + 审计	赊账订单闭环端到端可跑通，审计链完整
Phase 1	L1 路径 + 冲突恢复 + 补偿 + 澄清	L1 自动执行；CONFLICT 可恢复；补偿可回滚
Phase 2	评测（冻结上下文快照）+ 可选验证器 + 模型适配层	300–500 条样本；危险写入拦截率、误拒率、P95、成本可复现
Phase 3	行业插件 + 计量计费 + 角色细化	行业规则可插拔；套餐配额可执行

相关文档

- 13-trusted-execution-architecture.md — 可信执行架构详细设计（契约、MCP 工具链、preview/confirm/execute、评测基准）
- 06-system-architecture.md — 整体技术架构设计
- 03-mcp-tool-whitelist.md — MCP 工具白名单体系（工具命名与分类以 14 号为准）
- 04-business-engine.md — 统一业务引擎设计（规则系统已并入契约 Check）
- 08-risk-classification.md — 风险分级素材（等级模型已由 14 号裁决为 L0/L1/L2/Block）
- 09-fast-path.md — 快速通道与白名单（权限边界已由 14 号收窄）
- 10-dynamic-risk.md — 动态风险因子来源（已降级为 riskRules 输入）
- 11-fallback.md — 多级兜底策略（档位措辞待按本框架重写）
- 12-hook-system-design.md — Hook 系统（已收编为契约 Check 实现）
- 07-data-flow-design.md — 核心请求全流程（历史方案，保留作对比）
- 01-multi-agent-design.md — Multi-Agent 协同决策（历史方案）
- 02-openclaw-development.md — OpenClaw 内核定制（历史方案）
- 05-four-layer-safety.md — 六重防护机制（历史方案）